Module 2: Concept Mapping

How synonyms, abbreviations, typos and codes resolve to the same business concept

Learning objectives

By the end of this module, you should be able to:

1. Describe synonyms as **equivalence classes** and explain why transitivity needs guarding.
2. Measure surface-level similarity with **edit distance**.
3. Resolve ambiguous terms with a **softmax over candidate concepts**, and judge how uncertain the result is with **entropy**.
4. Expand abbreviations with a **noisy-channel (Bayesian) model**.
5. Link raw codes to concepts with **pointwise mutual information (PMI)**.
6. Measure **semantic distance** in an ontology graph with path length and information content.

2.1 The problem: many words, one meaning

In Module 1, we assumed a function C(·) that turns a question into a set of concepts. This module shows how that function can work.

The same business idea appears in many forms:

Form	Examples, all meaning *Revenue*
Synonym	sales, turnover, income from sales
Abbreviation	rev, REV, Rev.
Typo	revnue, reveune
Code	R01, ACCT_4000
Domain jargon	top line, bookings (sometimes)

The goal is a mapping from every **surface form** t to a **concept** c:

```
L : Surface forms → Concepts
L("sales") = L("rev") = L("revnue") = L("R01") = Revenue
```

The difficulty is that the mapping is often **ambiguous**. "Sales" can mean revenue, units sold, or the sales team. So we need L to output a probability distribution over concepts, not a single answer:

```
L(t) → P(c | t, context)
```

2.2 Equivalence classes

Definition

Define a relation ~ on surface forms: **$t_1 \sim t_2$ if both refer to the same concept**. For ~ to be an equivalence relation, it must be:

Property	Formal	Meaning
Reflexive	$t \sim t$	Every term means itself
Symmetric	$t_1 \sim t_2 \Rightarrow t_2 \sim t_1$	Synonymy goes both ways
Transitive	$t_1 \sim t_2 \wedge t_2 \sim t_3 \Rightarrow t_1 \sim t_3$	Synonyms of synonyms are synonyms

An equivalence relation **partitions** the vocabulary into disjoint classes:

```
[Revenue]  = { revenue, sales, turnover, rev, top line }
[Customer] = { customer, client, account holder, cust }
[Region]   = { region, territory, geo, area }
```

Every term belongs to exactly one class, and its concept is the class it lives in.

The transitivity trap

Real language is *not* perfectly transitive. Chaining loose synonyms causes **semantic drift**:

```
sales ~ revenue ~ income ~ earnings ~ salary
⇒ sales ~ salary   ✗
```

Fix 1: multiply confidences along a chain. Each link carries a confidence, and the confidence of a chain is the product of its links:

$$conf(t_1 \sim t_k) = \Pi_{i=1}^{k-1} conf(t_i \sim t_{i+1})$$

For example, 0.9 × 0.8 × 0.6 × 0.5 = **0.216**, which is too weak to trust.

Fix 2: only allow strict synonym edges to merge classes. Weaker relations such as *related-to* never merge classes. Instead they become distances (see 2.7).

2.3 Surface similarity: edit distance

Typos and spelling variants are caught by measuring how many character edits separate two strings.

Levenshtein distance

The minimum number of **insertions, deletions or substitutions** needed to turn string a into string b. It's computed with dynamic programming:

$$lev(i, j) = \min \begin{cases} lev(i-1, j) + 1, & \text{(delete)} \\ lev(i, j-1) + 1, & \text{(insert)} \\ lev(i-1, j-1) + 1[a_i \neq b_j] & \text{(substitute)} \end{cases}$$

Normalized similarity

$$sim_{lex}(a, b) = 1 - lev(a, b) / \max(|a|, |b|)$$

Example: "revnue" vs. "revenue"

- One insertion (the missing "e"), so lev = 1
- max length = 7
- sim_lex = 1 - 1/7 ≈ **0.857**

> **Normalization first.** Before comparing, apply cheap transforms such as lower-casing, trimming punctuation and reducing words to their stems ("revenues" → "revenue"). Many matches are exact after normalization.

Limitation: edit distance knows nothing about meaning. "cost" and "coast" are 1 edit apart. "revenue" and "turnover" are 7 edits apart (every letter of "turnover" must be replaced, and one extra letter added). This is why lexical similarity is only one input among several.

2.4 Resolving ambiguity: softmax over concepts

When a term could mean several concepts, we score each candidate and turn the scores into probabilities.

The model

```
P(c | t, Q) = exp( z(c) / T ) / Σ_{c'} exp( z(c') / T )

z(c) = s(t, c) + λ · ctx(c, Q)
```

- **s(t, c)** is how closely t matches c's known aliases, for example the maximum embedding cosine over the class [c].
- **ctx(c, Q)** is how well c fits the *other* concepts in the question.
- **λ** is how much context matters.
- **T** is the temperature. A low T makes the distribution sharper and more decisive; a high T makes it flatter and more cautious.

Context coherence

A concept fits better when it commonly appears with the other concepts already in the question:

$$ctx(c, Q) = \Sigma_{c_j \in C(Q) \setminus \{c\}} w(c, c_j)$$

Here $w(c, c_j)$ is how often c and c_j appear together in practice. For example, "Revenue" with "Region" is common, while "Sales Team" with "Quarter total" is less common.

Worked example

Question: *"Show sales by region last quarter."* The term is **"sales"**. Use λ = 1 and T = 1.

Candidate concept	s(t, c)	ctx(c, Q)	z(c)
Revenue	0.9	0.6	**1.5**
UnitsSold	0.7	0.5	**1.2**
SalesDepartment	0.8	0.1	**0.9**

Softmax:

```
e^1.5 = 4.482    e^1.2 = 3.320    e^0.9 = 2.460    Σ = 10.261

P(Revenue)         = 4.482 / 10.261 ≈ 0.44
P(UnitsSold)       = 3.320 / 10.261 ≈ 0.32
P(SalesDepartment) = 2.460 / 10.261 ≈ 0.24
```

Revenue wins, but **not by much**. How uncertain is this?

Measuring uncertainty: entropy

```
H(P) = − Σ_c P(c) · ln P(c)        maximum = ln(number of candidates)

H = −[0.44·ln 0.44 + 0.32·ln 0.32 + 0.24·ln 0.24]
  ≈ 0.362 + 0.365 + 0.342 = 1.07 nats

H_max = ln 3 ≈ 1.10   ⇒   normalized entropy ≈ 0.97   (very uncertain)
```

What new context does

Suppose the user adds *"…in dollars."* The unit "dollars" strongly fits Revenue, so ctx(Revenue) rises to 1.5 and z(Revenue) becomes 2.4.

```
e^2.4 = 11.023    Σ = 11.023 + 3.320 + 2.460 = 16.803

P(Revenue) ≈ 0.66    P(UnitsSold) ≈ 0.20    P(SalesDepartment) ≈ 0.15
```

> **Key insight:** a single word was ambiguous, but the surrounding words resolved it. Context is a form of evidence.

Decision rule

Accept the top concept only if it is both **confident** and **clearly ahead** of the runner-up:

```
accept c₁   if   P(c₁) ≥ τ   and   P(c₁) − P(c₂) ≥ δ
otherwise   →  ask a clarifying question, or state the assumption made
```

With τ = 0.6 and δ = 0.2, the first case (0.44, margin 0.12) leads to clarification. The second case (0.66, margin 0.46) is accepted.

2.5 Abbreviations: the noisy-channel model

Think of an abbreviation as a **corrupted message**. Someone had a full form f in mind and sent a compressed version a. We want to recover f.

Bayes' rule

```
P(f | a) ∝ P(a | f) · P(f)
           ──────────  ────
           "could f    "how common
           shorten     is f here?"
           to a?"
```

Likelihood P(a | f): how plausible the shortening is

This is high when a matches f's structure:

Pattern	Example	Plausibility
Initials	NA ← **N**orth **A**merica	High
Prefix	Rev ← **Rev**enue	High
Consonant skeleton	Mgmt ← **M**anag**e**m**e**n**t**	Medium
Subsequence	Qty ← **Q**uan**t**i**y**	Medium
No structural match	XYZ ← Revenue	≈ 0

Prior P(f): how common the full form is

The prior should depend on **context**, not just global frequency. That is where most of the disambiguation happens.

Worked example: "NA"

Candidates: North America, Not Applicable, Net Amount.

Case A: no context. "What is NA?"

f	P(a | f)	P(f)	Product	Posterior
North America	0.9	0.3	0.27	0.27/0.81 ≈ **0.33**
Not Applicable	0.8	0.5	0.40	0.40/0.81 ≈ **0.49**
Net Amount	0.7	0.2	0.14	0.14/0.81 ≈ **0.17**

The result is ambiguous: "Not Applicable" leads narrowly.

Case B: context is a region column. "Revenue for NA vs EMEA." The neighbouring term "EMEA" is a region, so the prior shifts strongly.

f	P(a | f)	P(f | region context)	Product	Posterior
North America	0.9	0.85	0.765	0.765/0.88 ≈ **0.87**
Not Applicable	0.8	0.10	0.080	0.080/0.88 ≈ **0.09**
Net Amount	0.7	0.05	0.035	0.035/0.88 ≈ **0.04**

The result is now clear: **North America**.

> **Lesson:** the likelihood barely changed. The context-dependent prior did all the work. Never resolve an abbreviation in isolation.

> **Data-quality note:** "NA", "N/A", "Unknown" and empty strings often appear as **placeholder values** in data. A good mapping keeps them in a separate "missing or unknown" class rather than treating them as a real category.

2.6 Codes in the data: pointwise mutual information

Internal codes such as "R01", "ACCT_4000" or "ST-07" often have no linguistic structure at all. We can only learn what they mean from **where they appear**.

PMI

Measures whether a code v and a concept c appear together **more than chance would predict**:

$$PMI(v, c) = \log_2 \left[P(v, c) \,/\, \left(P(v) \cdot P(c) \right) \right]$$

PMI value	Interpretation
> 0	Appear together more than expected: likely related
≈ 0	Independent
< 0	Appear together less than expected: likely unrelated

Worked example

Suppose we examine 1,000 documented field contexts, such as column descriptions, report labels and filters:

- Code "R01" appears in 50 of them, so P(v) = 0.05.
- Concept *Region* appears in 200, so P(Region) = 0.20.
- They appear together in 40, so P(v, Region) = 0.04.
- Concept *Revenue* appears in 300, so P(Revenue) = 0.30.
- "R01" and Revenue appear together in 5, so P(v, Revenue) = 0.005.

$$PMI(R01, Region) = \log_2\left(0.04 \,/\, (0.05 \times 0.20) \right) = \log_2\left(0.04 / 0.010 \right) = \log_2 4 = +2.0 \text{ bits}$$

$$PMI(R01, Revenue) = \log_2\left(0.005 \,/\, (0.05 \times 0.30) \right) = \log_2\left(0.005 / 0.015 \right) = \log_2 0.333 \approx -1.6 \text{ bits}$$

Conclusion: "R01" is a **region code**, not a revenue code, even though both start with "R".

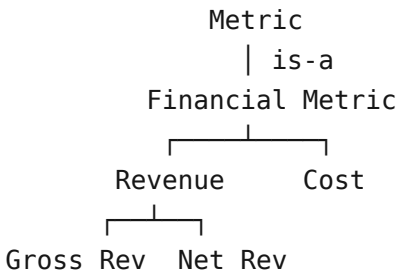
> **Positive PMI (PPMI).** In practice, negative values are noisy when counts are small, so they are often clipped: PPMI = max(0, PMI).

> **Rare-event caution.** PMI exaggerates relationships for very rare codes. Common remedies are to require a minimum count, or to add a smoothing constant k to every count ($P \rightarrow (\text{count} + k) / (N + k \cdot |V|)$).

2.7 Semantic distance in an ontology

Some concepts aren't synonyms but are still **close**. "Gross revenue" and "net revenue" are siblings. "Revenue" and "cost" are both financial metrics, but they are opposites in practice. An ontology graph captures this structure.

The ontology as a graph



Edges are **typed**, and each type has a traversal cost:

Edge type	Cost
synonym-of	0 (same concept)
is-a / parent-of	1
part-of	1.5
related-to	2

Method 1: Path-based similarity

$dist(c_1, c_2)$ = weighted shortest-path length (e.g. Dijkstra's algorithm)
$sim_{path}(c_1, c_2) = \exp(-\mu \cdot dist(c_1, c_2))$

With μ = 0.5:

Pair	Path	dist	sim_path
Gross Rev ↔ Net Rev	Gross → Revenue → Net	2	e^(-1.0) ≈ **0.37**
Gross Rev ↔ Cost	Gross → Revenue → Financial → Cost	3	e^(-1.5) ≈ **0.22**

Method 2: Information-content similarity (Lin)

The path method treats every edge equally. But two concepts that share a **specific** ancestor (such as Revenue) are more alike than two that share only a **generic** ancestor (such as Financial Metric).

Information content: IC(c) = -ln P(c), where P(c) is the probability that a random mention falls under c or any concept below it. Specific concepts are rare, so they have high IC.

$sim_{Lin}(c_1, c_2) = 2 \cdot IC(lcs(c_1, c_2)) / (IC(c_1) + IC(c_2))$

Here lcs is the *lowest common subsumer*: the most specific ancestor the two concepts share.

Assume the following probabilities:

Concept	P(c)	IC = -ln P
Financial Metric	0.50	0.693
Revenue	0.20	1.609
Cost	0.25	1.386
Gross Rev	0.05	2.996
Net Rev	0.08	2.526

```
sim_Lin(Gross, Net)  = 2(1.609) / (2.996 + 2.526) = 3.219 / 5.521 ≈ 0.58    (lcs = Revenue)
sim_Lin(Gross, Cost) = 2(0.693) / (2.996 + 1.386) = 1.386 / 4.382 ≈ 0.32    (lcs =
Financial Metric)
```

Both methods agree on the order, but Lin separates the two pairs more sharply.

Why not just use embeddings?

Embedding models learn from **which words appear in similar contexts**. "Revenue" and "cost" appear in very similar sentences ("Q3 ___ rose 5%"), so their embeddings are often **very close**. An ontology encodes that they are **distinct, even opposite** concepts.

```
cos(φ("revenue"), φ("cost")) ≈ high      ← embeddings: "these are alike"
sim_Lin(Revenue, Cost)       ≈ low       ← ontology:   "these differ"
```

> **Lesson:** embeddings measure **relatedness**, while ontologies measure **sameness**. Concept mapping needs both.

2.8 Putting it together: a unified mapping score

Every signal from this module feeds a single score per (term, concept) pair:

```
z(t, c) = w₁ · cos( φ(t), φ(c) )                    ← semantic relatedness (embeddings)
        + w₂ · max_{t' ∈ [c]} sim_lex(t, t')         ← spelling / typo match (2.3)
        + w₃ · log P(t | c)                          ← abbreviation fit (2.5)
        + w₄ · PPMI_norm(t, c)                       ← co-occurrence for codes (2.6)
        + w₅ · Σ_{cⱼ ∈ C(Q)} sim_onto(c, cⱼ)         ← ontology coherence with the question
(2.7)
        + w₆ · log P(c)                              ← prior: how common or curated c is
```

Then:

```
P(c | t, Q) = softmax_c( z(t, c) / T )                                    (2.4)
ĉ(t) = argmax_c P(c | t, Q)   if  P(ĉ) ≥ τ  and  P(ĉ) − P(c₂) ≥ δ
       else  → clarify or state the assumption
```

Which signal helps most for each surface form?

Surface form	Dominant signals
Synonym ("turnover")	w_1 embeddings, w_5 ontology
Typo ("revnue")	w_2 edit distance
Abbreviation ("NA")	w_3 noisy channel plus context prior
Internal code ("R01")	w_4 PMI
Ambiguous word ("sales")	w_5 context coherence plus clarification threshold

2.9 Common pitfalls

Pitfall	Symptom	Fix in the math
Unbounded transitivity	"sales" ends up equal to "salary"	Multiply chain confidences; only strict synonyms merge classes
Relying only on embeddings	"Revenue" confused with "cost"	Add ontology similarity (w_5)
Abbreviations without context	"NA" read as Not Applicable in a region column	Make the prior context-dependent: P(f | context)
Raw PMI on rare codes	Spurious strong links	Minimum counts, smoothing, PPMI
Forcing a single answer	Silent wrong guesses on ambiguous terms	Entropy check plus τ/δ decision rule
Treating placeholders as values	"N/A" counted as a real category	Keep a separate "missing or unknown" class

Check your understanding

1. **Compute:** what is the normalized Levenshtein similarity between "custmer" and "customer"?
2. **Compute:** three candidate concepts have scores z = (2, 1, 0) with T = 1. What are their softmax probabilities?
3. **Compute:** a code appears in 10% of contexts, a concept in 10%, and both together in 2%. What is the PMI in bits?
4. **Reason:** in the "sales" example, what happens to the probabilities if the temperature T is raised from 1 to 10?
5. **Reason:** why might two concepts with high embedding similarity still have low Lin similarity?

<details>
<summary>Answers</summary>

1. One insertion, so lev = 1, and the max length is 8. The similarity is 1 - 1/8 = **0.875**.
2. $e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$. The sum is 11.107. The probabilities are ≈ **(0.665, 0.245, 0.090)**.
3. $\log_2(0.02 / (0.1 \times 0.1)) = \log_2(2) =$ **+1 bit**. They co-occur twice as often as chance.

4. The scores are divided by 10, giving 0.15, 0.12 and 0.09, so the distribution becomes **nearly uniform** (≈ 0.34, 0.33, 0.32). The system becomes much less decisive and would almost certainly ask for clarification.
5. Embeddings capture *contextual relatedness*: the words appear in similar sentences. Lin captures *taxonomic sameness*: the concepts share a specific ancestor. Opposites such as revenue and cost are contextually related but taxonomically distinct.

</details>

Key takeaways

- Synonyms form **equivalence classes**, but transitivity must be **confidence-weighted** to prevent drift.
- **Edit distance** catches typos, **noisy-channel Bayes** expands abbreviations, and **PMI** decodes opaque codes.
- **Softmax plus entropy** turns ambiguous terms into calibrated probabilities, and the τ/δ thresholds decide when to ask instead of guess.
- **Context-dependent priors** do most of the disambiguation work.
- **Embeddings measure relatedness, while ontologies measure sameness.** Robust mapping combines both.

Coming next: Module 3, Inferring Implicit Relationships

The user asked about "customers" and "revenue", but those live in different tables with no stated link. How does a system figure out how to connect them? We'll cover schema graphs, minimum-cost join paths, Steiner trees, value-overlap evidence (Jaccard, containment, cardinality), latent default filters, and transitive inference.